



VAS VÁRMEGYEI SZAKKÉPZÉSI CENTRUM
HORVÁTH BOLDIZSÁR
KÖZGAZDASÁGI ÉS INFORMATIKAI
TECHNIKUM

A 5 0613 12 03 számú Szoftverfejlesztő és –tesztelő vizsgaremek

A szoftveralkalmazás dokumentációja

Készítették:

FEHÉR ÁGOSTON
FÜLÖP LÁSZLÓ ANDRÁS
MÜLLER BÁLINT

SZOMBATHELY

2026

Bevezetés

Ez a dokumentáció a Szoftverfejlesztő és -tesztelő képzés keretein belül készített vizsgamunkánkat mutatja be. Egy naptárkezelő szoftvert hoztunk létre, amely minden korosztály számára egyszerűbbé és hatékonyabbá teszi az eseményeik számontartását és egymás közötti megosztását. Ebben a dokumentációban megpróbáljuk a lehető legalaposabban ismertetni a fejlesztési folyamatot, az alkalmazás működését és a fejlesztéshez használt eszközöket. Emellett részletes leírásokkal bemutatjuk a program funkcióit és segítséget nyújtunk az oldal használatában.

Szoftver célja

A szoftverünk célja egy olyan letisztult, szabványoknak megfelelő, felhasználóbarát felületet biztosító alkalmazás, amellyel gyorsan és egyszerűen tudják vezetni a felhasználók az eseményeiket, továbbá megoszthatják ezeket a rokonokkal, és/vagy az ismerősökkel is. Ismerősi körökben észleltük, hogy nehezen tudják összeegyeztetni a programokat, mivel nem ismerik a másik napirendjét, és ezt a naptárjukban nem látják. Ezért a jelen alkalmazással a célunk az volt, hogy a felhasználóink egyszerűen tudják egymással megosztani az eseményeiket és programjaikat.

Felhasználói felületek

A projektünket úgy terveztük, hogy reszponzív¹ legyen, tehát mind mobilon, tableten, asztali számítógépen vagy laptopon egy letisztult felhasználói élményt nyújtson. Figyeltünk arra, hogy az oldal elemei ne csússzanak egymásba és ne takarják el egymást akkor sem, ha csökkentjük a képernyő méretét. Ügyeltünk arra is, hogy a kisebb képernyőkön is jól láthatóak legyenek, valamint arra is, hogy a mobil kijelzőkön is felhasználóbarát legyen az oldal.

¹ Reszponzív: Az oldal elemei reagálnak a teljes oldal méretének változására, és attól függően változik azon elemek mérete/elrendezése is

Készítési folyamat

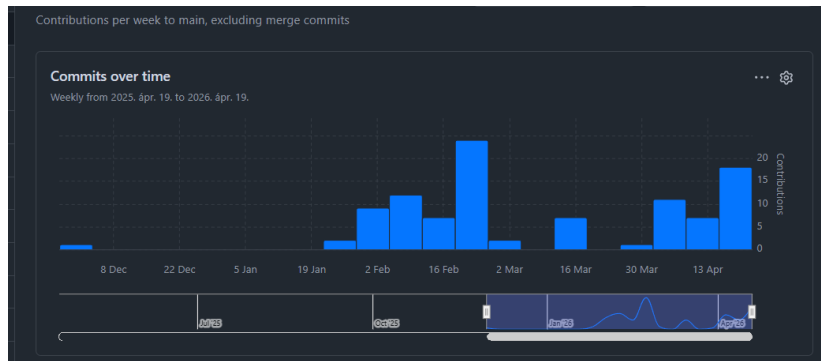
Feladatok felosztása:

A szoftver elkészítése több lépésből álló hosszú folyamat, amelynek során problémák adódhatnak, melyeket meg kell oldania a csapattagoknak. A feladatokat az egyenlő munkamegosztás érdekében felosztottuk magunk között, majd a projekt céljai és követelményei alapján határoztuk meg a tennivalókat. A munkafolyamat felosztása közben figyelembe vettük a feladatok bonyolultságát, nehézségét és időigényességét. Ezek mellett figyeltünk arra is, hogy a teendők a csapattagok szakértelmének megfelelően kerüljenek kiosztásra. A részfeladatok elvégzése után a modult mindenki megnézte és ellenőrizte is.

Feladat	Készítette
Projekt – Alaptervek	Fülöp László, Fehér Ágoston
Adatbázis – ER Modell	Fülöp László
Adatbázis – Relációs Modell	Fülöp László
Adatbázis – Modellezés	Fehér Ágoston
Adatbázis – Mintaadatok	Müller Bálint
Backend – AdvancedInfo kontroller	Fehér Ágoston
Backend – Admin kontrollerek	Fehér Ágoston, Müller Bálint
Backend – Users kontroller	Fehér Ágoston, Müller Bálint
Backend – Login kontroller	Fehér Ágoston, Fülöp László
Backend – Modellek	Müller Bálint
Backend – Unit tesztek	Fülöp László, Fehér Ágoston
Frontend – Bejelentkező/Regisztráció felület	Fülöp László
Frontend – „LoginModule.jsx” háttér logika	Fülöp László, Fehér Ágoston

Frontend – Főoldal	Fülöp László
Frontend – „About Us” oldal	Fülöp László
Frontend – „Contact Us” oldal	Fülöp László
Frontend – Beállítások oldal	Fehér Ágoston
Frontend – Értesítések oldal	Fehér Ágoston
Frontend – Naptár havi nézet	Müller Bálint
Frontend – Naptár heti nézet	Müller Bálint
Frontend – Naptár napi nézet	Müller Bálint
Frontend – Naptár nézet háttér logikája	Fehér Ágoston
Frontend – Esemény nézet	Fehér Ágoston, Müller Bálint
Frontend – Esemény hozzáadása	Fehér Ágoston, Müller Bálint
Frontend – Dizájn	Fülöp László, Fehér Ágoston
Frontend – CSS fájlok	Fülöp László
Frontend – JS fájlok	Fehér Ágoston
Frontend – Reszponzivitás	Fülöp László, Fehér Ágoston
Frontend – Manuális tesztelés	Fehér Ágoston, Fülöp László, Müller Bálint
Dokumentáció	Fehér Ágoston, Fülöp László, Müller Bálint
Dokumentáció – Képek készítése	Fülöp László, Müller Bálint
Dokumentáció – Átnézése és ellenőrzése	Fehér Ágoston
Koordinátor	Fehér Ágoston

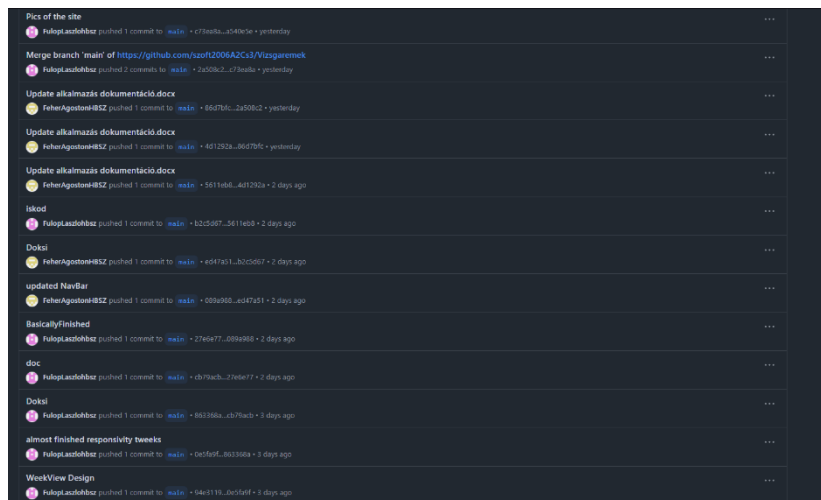
1. táblázat: A feladatelosztások



1. képleírás: A GitHub-on végzett munka gráfja

Verziókövetés:

A szoftverünk fejlesztése során a közös munka folyamán a változtatásaink egyszerű nyomon követhetőségének érdekében a „GitHUB” verziókezelő rendszert használtuk. A „Repository”-t még egyik tanárunk hozta létre év elején, amelyben egész idő alatt dolgoztunk. Erről a „repository”-ről alább csatolunk pár képet is, mert amelyet véglegesként leadtunk nem egyezik meg ezzel. Sajnos az első „repository”-nk nem volt publikus, mert nem rendelkezünk rendszergazdai jogosultságokkal. Ezért a könnyű és gyors megosztás érdekében, egy új „repository”-ba töltöttük fel a végleges munkánkat.



2. képleírás: A GitHub-on végzett commit-ok egy része

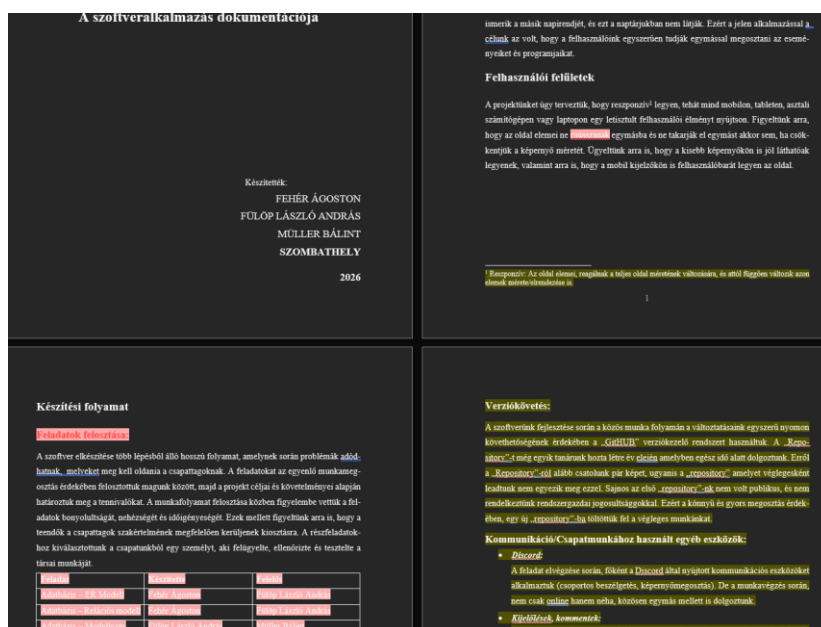
Kommunikáció/Csapatmunkához használt egyéb eszközök:

Discord:

A feladat elvégzése során főként a Discord által nyújtott kommunikációs eszközöket alkalmaztuk (csoportos beszélgetés, képernyőmegosztás). De a munkavégzés során nem csak online, hanem néha közösen, egymás mellett is dolgoztunk.

Kijelölések, kommentek:

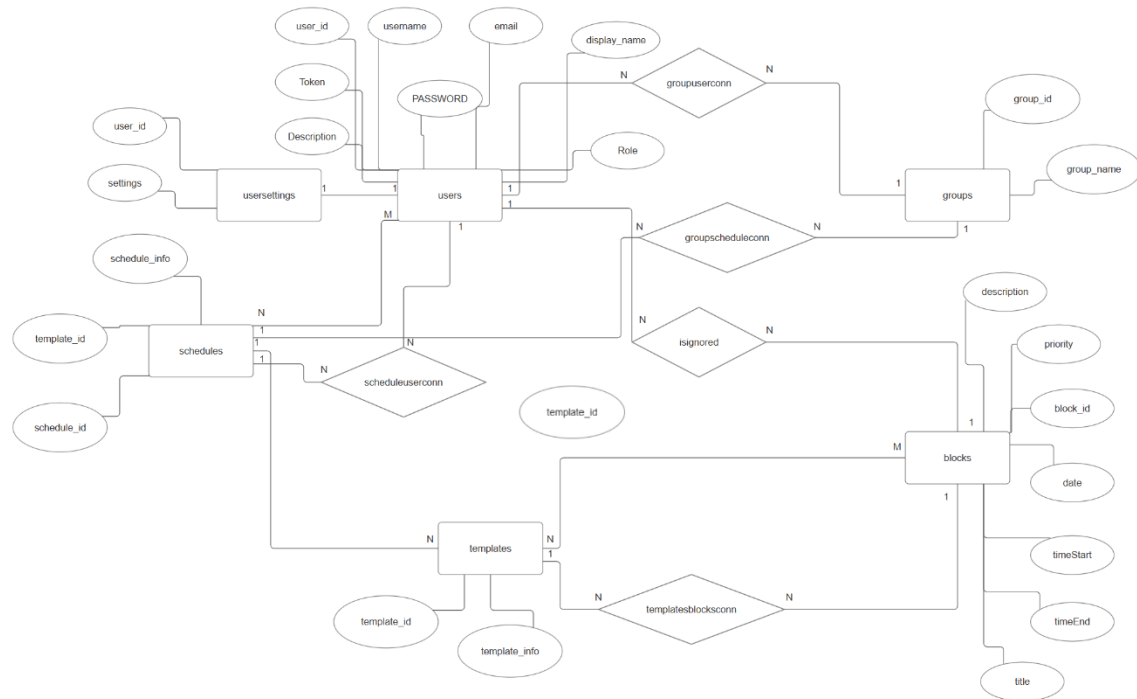
A dokumentum, és a projekt kódjának szerkesztése során gyakran hagyunk akár mások, akár saját magunk emlékeztetése érdekében kijelölt részeket/kommenteket, amelyek rövid, egyértelmű jelentéssel bírtak. Például a dokumentum írása során a még hiányos részeket piros kijelöléssel, a már befejezett – de még nem ellenőrzött – részeket pedig sárga kijelöléssel láttuk el.



3. képleírás: Egy részlet a dokumentumról a készítés folyamán

Adatbázis

Az adataink tárolásához – a XAMPP Control Panel alkalmazás segítségével – egy MySQL adatbázist hoztunk létre. Ez az adatbázis felelős a fő adatok tárolásáért, mint például a felhasználók egyedi azonosítóinak és tokenjeinek tárolása. Ez segít továbbá még a csoportokon belüli jogosultság kezelés megvalósításában is.



1. ábra: Az adatbázis ER modellje

Adatbázis táblái és rövid összefoglalásuk:

„blocks” tábla:

- **block_Id** A blokk azonosítója
- **date** A blokk dátuma (éééé-hh-nn)
- **description** Az adott blokk leírása
- **priority** Az adott blokk prioritása
- **timeEnd** Mikor ér véget a blokk (percben)
- **timeStart** Mikor kezdődik a blokk (percben)
- **title** A blokk címe

Ez a táblázat tartalmazza az esemény blokkokkal kapcsolatos információit.

„groups” tábla:

- group_Id A csoport azonosítója
- group_name A csoport neve

Ez a tábla tartalmazza a csoportokat és az azokkal kapcsolatos információkat.

„groupscheduleconn” tábla:

- group_Id A csoport azonosítója
- schedule_Id A schedule azonosítója

Ez a tábla tartalmazza a csoportok és naptárak közötti egy – több kapcsolatokat.

„groupuserconn” tábla:

- group_Id A csoport azonosítója
- permission A felhasználó jogosultsága a csoportban
- user_Id A felhasználó azonosítója

Ez a tábla tartalmazza a felhasználók és csoportok közötti több – több kapcsolatot, és a csoporthoz kapcsolódó felhasználók jogosultságait.

„isignored” tábla:

- block_id A blokk azonosítója
- user_id A felhasználó azonosítója

Ez a tábla tartalmazza, hogy a felhasználók mely blokkok értesítéseit mellőzik, hogy ne kapjanak értesítést.

„schedules” tábla:

- schedule_id A schedule azonosítója
- schedule_info A schedule információi
- template_id A sablon azonosítója

Ez a tábla tartalmazza a naptárakat és ezek információit.

„scheduleusersconn” tábla:

- schedule_id A schedule azonosítója
- user_id A felhasználó azonosítója

Ez a tábla tartalmazza a felhasználók és személyes naptáraik közötti egy – több kapcsolatot.

„templates” tábla:

- template_id A sablon azonosítója
- template_info A sablon információi

Ez a tábla tartalmazza a sablonokat és ezek információit.

„templatesblocksconn” tábla:

- block_id A blokk azonosítója
- template_id A sablon azonosítója

Ez a tábla tartalmazza a sablonok és blokkok közötti egy – több kapcsolatot.

„users” tábla:

- Description A felhasználó leírása
- display_name A felhasználó megjelenített neve
- email A felhasználó e-mail címe
- PASSWORD A felhasználó jelszava – titkosítva
- Role A felhasználó jogosultsága
- Token A felhasználó token-je
- username A felhasználó felhasználóneve
- user_id A felhasználó azonosítója

Ez a tábla tartalmazza a felhasználókat és az azokkal kapcsolatos információkat.

„usersettings” tábla:

- settings A beállításokkal kapcsolatos információk
- user_id A felhasználó azonosítója

Ez a tábla tartalmazza a felhasználó beállításainak információit.

Felhasználói felület megtervezése:

Felhasználói felület felosztása:

A felhasználói felület a bejelentkezéstől függően két fő részből áll. Amennyiben nincs bejelentkezett felhasználó, akkor egy alapvető ismertető főoldal várja az új felhasználókat. Itt tudnak olvasni a szoftver által nyújtott funkciókról és ennek a működésével kapcsolatos információkról. A másik fő egység a belépett felhasználók számára biztosítja az elérhető funkciókat és az azok könnyű eléréséért tervezett második számú főoldalt. Ezen a másodlagos főoldalon a különböző funkciókhoz tartozó elérési hivatkozásokat szimbolizáló gombokat találjuk, valamint az egyéb, a bejelentkezett felhasználóinkhoz tartozó információkat.

Felhasználói felület arculata:

A felhasználói felülettel törekedtünk a modernsége és az egyszerűsége. A mai weboldalak szabványai alapján terveztük meg és készítettük el az alkalmazást. Próbáltuk elérni, hogy egy új felhasználó számára is könnyű legyen az oldal használata, és ennek vizuális megjelenése is vonzó legyen.

A legfeltűnőbb elemei egy weboldalnak: az elemek, a háttérszín, illetve a betűtípus. Az utóbbi években előtérbe kerültek a sötét témájú oldalak, a legtöbb ember ezeket preferálja, mivel a szemnek ez nem annyira fárasztó és megterhelő, főleg egy kevésbé megvilágított helyiségben. Ezen okok miatt választottuk a sötétlila színt az oldal fő színének. Ezzel ellenében gondoltunk azokra is, akik a világos oldalakat részesítik előnyben. Emiatt a beállításkor lehet választani a világos és sötét témák között. A világos témában a világoslila és a világoszöld szín a domináns.

A különböző oldalak megtervezésénél a rezponzivitást is figyelembe vettük. Minden oldal az ablak átméretezését követően alkalmazkodik ehhez, sőt egyes oldalak a design érdekében még az elrendezésüket is megváltoztatják.

Szoftveregységek és -komponensek leírásai

Frontend:

A frontend React.js keretrendszer 19.2.3-as verziója segítségével készült a Vite 7.3.1-es verzióján belül a komponensekre bontott oldalrendezés és a beépített eszközök előnyeit kihasználva. Ennek köszönhetően könnyen átlátható és egyben később is egyszerűen bővíthető új funkciókkal. Kifejezetten hasznos volt a fejléc és a naptárnézet létrehozásában, így ezt nem kellett minden oldalon újraalkotni, hanem csak a komponens szükséges kódját kellett beilleszteni. A szoftver elkészítésében a React saját modulkönyvtára is nagy szerepet játszott. Meghívott modulok:

- react-dom:
-a React hivatalos csomagja, a React komponensek DOM-ba való rendelkezéséért felel.
- react-router-dom:
-lehetővé teszi a többoldalas alkalmazások készítését navigációval, útvonalkezeléssel és dinamikus URL-ekkel.

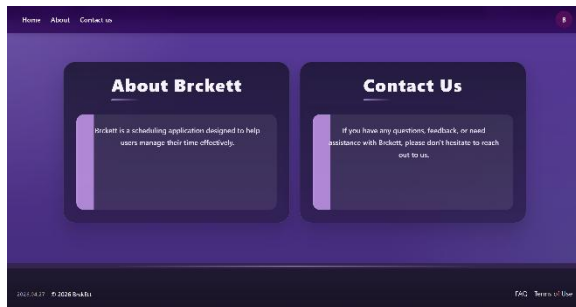
CSS:

A modern kinézetet CSS segítségével oldottuk meg. Minden komponens egy saját CSS fájlal rendelkezik (*komponens neve.css*), amely csak a saját komponenséhez tartozó HTML elemeket formázza. Ezeken felül alkalmaztunk egy-két globálisan is használt CSS stílusbeállítást is, főként a sötét/világos témák színénél, melyet CSS változók segítségével valósítottunk meg.

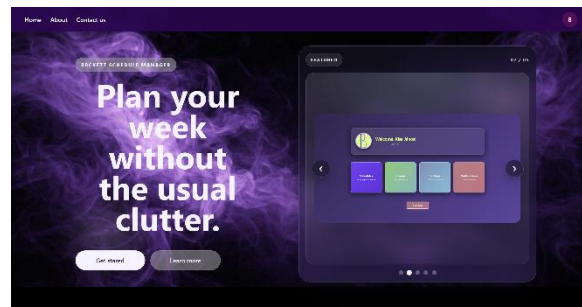
Frontend komponensek és feladataik:

FrontPage.jsx

Ez a komponens felelős a főoldal megjelenítéséért. Egy statikus, információt közlő oldalrész jelenít meg. Az oldal elején a logó, a mottó és egy karusszel található. Utána néhány rövid leírást és hivatkozásokat tartalmaz, amely a részletesebb leíró oldalakra, valamint a bejelentkezési ablakra irányít át. Ezek mellett még tartalmaz egy lábléceket, amelyről megjeleníthetők a „FAQ”² listája, vagy a „Terms of Use”³.



4. képleírás: Az oldal alsó része



5. képleírás: Az oldal felső része

ClickAnimation.jsx

Ez a komponens felelős a kattintást követő animációval kapcsolatos számolások és azok megjelenítéséért. Egy folyamatosan látható „Canvas”⁴ HTML elemet helyez minden más HTML elem fölé, ezáltal láthatóvá téve az animált effektet bárhol az alkalmazás felületén.

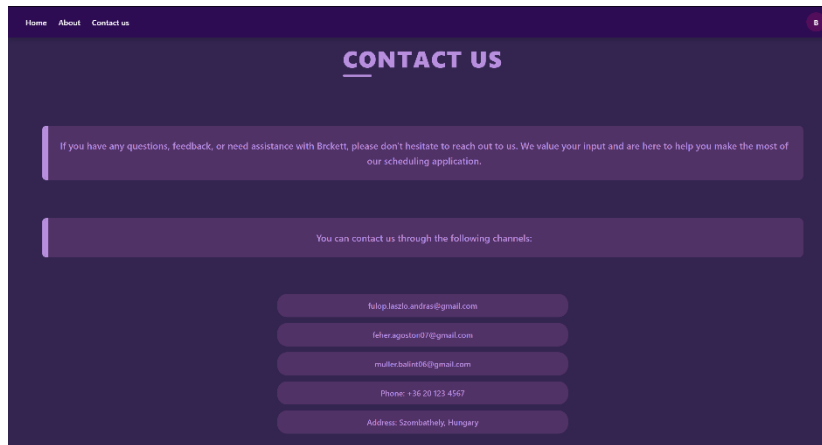
² FAQ: Gyakran ismételt kérdések

³ Terms of Use: Felhasználói feltételek

⁴ Canvas: Egy vásznat megjelenítő HTML elem, amelyre vektorgrafikusan tudunk rajzolni

ContactUs.jsx

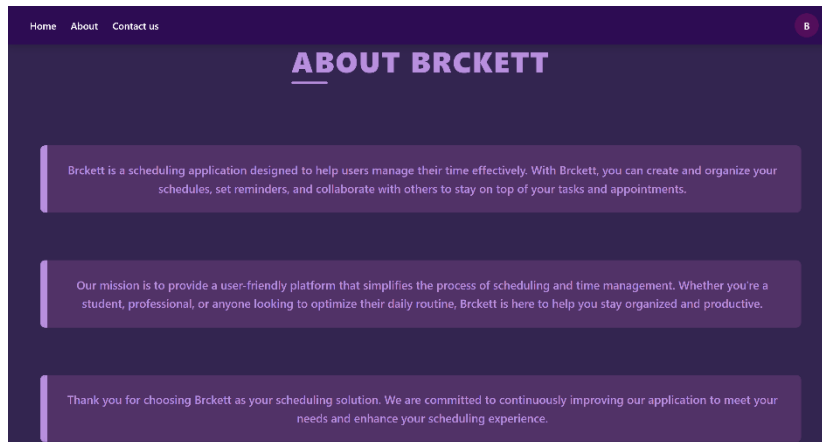
Ez a komponens felelős a „Contact Us”⁵ oldal megjelenítéséért. Egy statikus, információt közlő oldalrészlet jelenít meg. Az elérhetőségekkel és egyéb kapcsolatfelvételi információkkal kapcsolatos adatokat tartalmazza.



6. képleírás: A "Contact Us" oldal előnézete

About.jsx

Ez a komponens felelős az „About Us”⁶ oldal megjelenítéséért. Egy statikus, információt közlő oldalrészlet jelenít meg. A szoftver által nyújtott funkciók rövid bemutatását tartalmazza.



7. képleírás: Az "About Us" oldal előnézete

⁵ Contact Us: A kapcsolatfelvételi adatainkat tartalmazó oldal

⁶ About Us: A szoftver működését röviden leíró tájékoztató oldal

EventView.jsx

Ez a komponens felelős az adott naphoz tartozó blokkok megjelenítéséhez. A dátumra szűrve a szülőkomponenstől kapott blokkok listájából megjeleníti a kiválasztott naphoz tartozó eseményeket, ezáltal lehetőséget ad azok szerkesztésére/törlésére vagy akár új események létrehozására. A szerkesztés és létrehozás műveletekhez az „EventAdd.jsx” komponenst használja, amelynek az adott művelettel kapcsolatos alapértelmezett adatait egy „property”⁷ mezőn keresztül adja át. Ez a tulajdonság alapértelmezetten egy üres, adatok nélküli blokkstruktúra, amely egy új objektum létrehozása esetében látható. Amennyiben egy már meglévő blokkot szeretnénk szerkeszteni, akkor ez a „property” mező a kitöltendő mezők helyére beilleszti a kiválasztott blokk adatait.

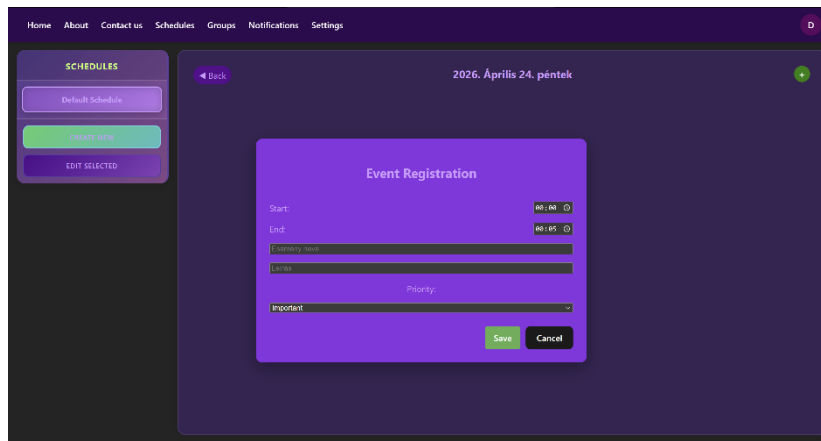


8. képleírás: Az adott nap eseményeit részletező oldal előnézete

⁷ property: Komponenseknek átadható információk egy általunk létrehozott mezőn keresztül

EventAdd.jsx

Ez a komponens egy felugró ablak megvalósításáért felelős. Ennek segítségével tudunk létrehozni és egyben szerkeszteni is már meglévő vagy új eseményeket egy naptárhoz, amelyeket a szülőkomponenstől kapott azonosító alapján csatol a „Backend”⁸ egy „Schedule”⁹-hoz. Ezeken felül a komponens a bemeneti idő mezőt (óó:pp) automatikusan átváltja percekké, amelyeket tárol az adatbázis. Emellett hibakezelést is végez annak érdekében, hogy a blokk ne lehessen 5 perc hosszúságúnál rövidebb, és ne érhesen előbb véget, mint ahogy elkezdődött.

The screenshot shows a web application interface with a dark theme. At the top, there is a navigation bar with links: Home, About, Contact us, Schedules, Groups, Notifications, and Settings. A user profile icon labeled 'D' is in the top right. On the left, a sidebar contains a 'SCHEDULES' section with buttons for 'Default Schedule', 'CREATE NEW', and 'EDIT SELECTED'. The main content area displays a date '2026. Április 24, péntek' and a 'Back' button. Centered is a white 'Event Registration' dialog box. It contains input fields for 'Start' (set to 00:00) and 'End' (set to 00:05), both with clear icons. Below these are text inputs for 'Location/Note' and 'Label'. A 'Priority' dropdown menu is set to 'Important'. At the bottom right of the dialog are 'Save' and 'Cancel' buttons.

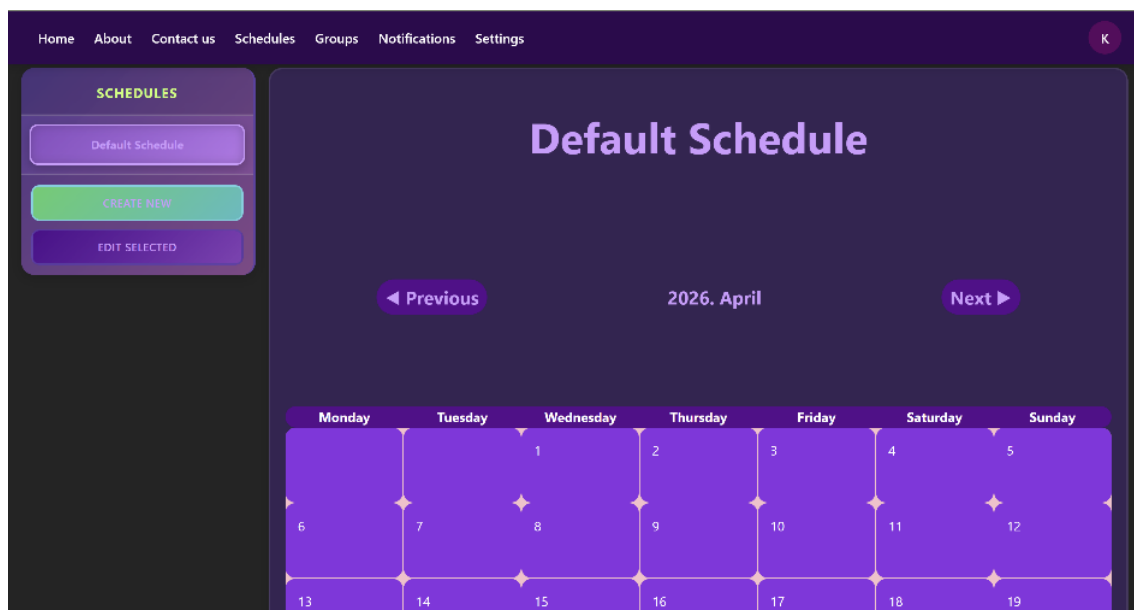
9. képleírás: Az új események létrehozását kezelő párbeszédablak

⁸ Backend: A szoftver háttérfolyamatait kezelő rész, amely az adatbázissal is tartja a kapcsolatot

⁹ Schedule: Egy naptár egyedítípust képviselő objektum

Calendar.jsx

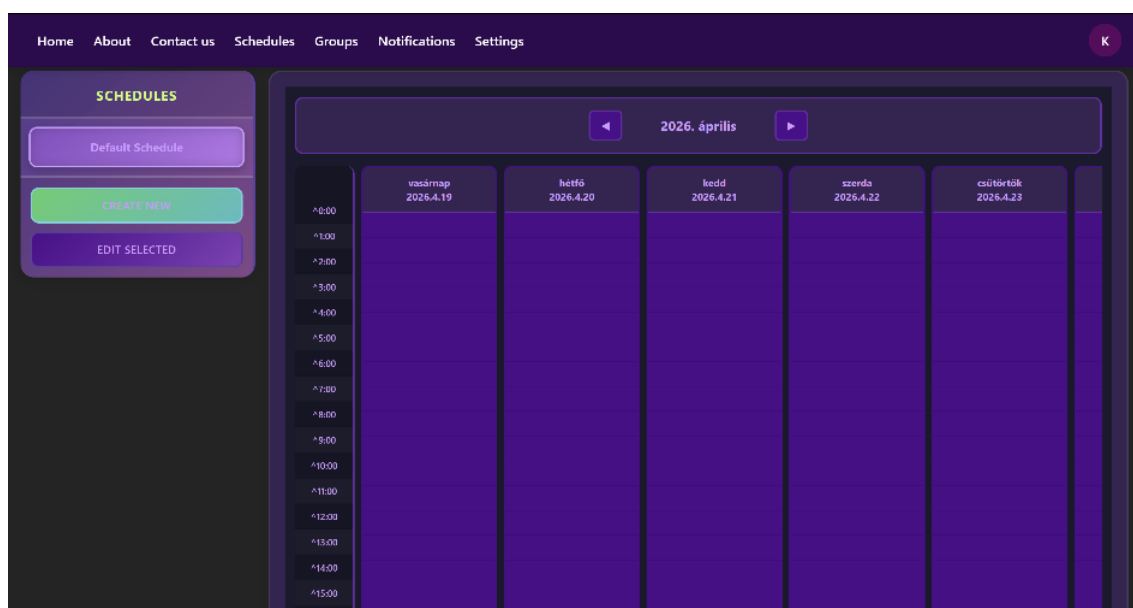
Ez a komponens felelős a havi felbontású naptár megjelenítéséhez, és minden vele kapcsolatos számításhoz. Megjeleníti a teljes hónap minden dátumát az annak megfelelő nap oszlopa alatt. Ezekre a mezőkre kattintva nyitható meg az adott naphoz tartozó eseményszerkesztő. Továbbá minden mezőhöz, amelyhez ha tartozik esemény, abban az esetben egy kör ikonnal jelzi ezt a mező alján. Amennyiben a jelzések nem férnek ki a mezőben, akkor egy „+” jellel jelöli azt. A fejlécben kijelzett aktuális dátumra kattintva vissza lehet lépni arra az intervallumra, amely tartalmazza a mai napi dátumot.



10. képleírás: A naptár havi elrendezésű előnézete

CalendarWeekView.jsx

Ez a komponens felelős a heti felbontású naptár megjelenítéséért és minden vele kapcsolatos számításért. Megjeleníti a hét napjait a fejlécen, amely alatt egy percekre bontott „Grid”¹⁰ segítségével percre pontosan jeleníti meg az adott naphoz tartozó blokkokat oszlopos formában. Az oszlopokra kattintva az adott nap eseményszerkesztőjét nyithatjuk meg. A responszivitás érdekében a napokat egy görgethető mező tartalmazza. A vizualizálás megkönnyítése a görgethető mező bal oldalán egy órákat felsoroló oszlop található. Ez az oszlop görgetéstől függetlenül mindig látszik. Segít abban is, hogy egy adott oszlop egy blokkot szimbolizáló eleme mellé görgetve le tudjuk olvasni annak kezdeti- és végidőpontját pontosan, mivel a címkék méretarányosan helyezkednek el a blokkokhoz képest. A fejlécben kijelzett aktuális dátumra kattintva vissza lehet lépni arra az intervallumra, amely tartalmazza a mai napi dátumot.

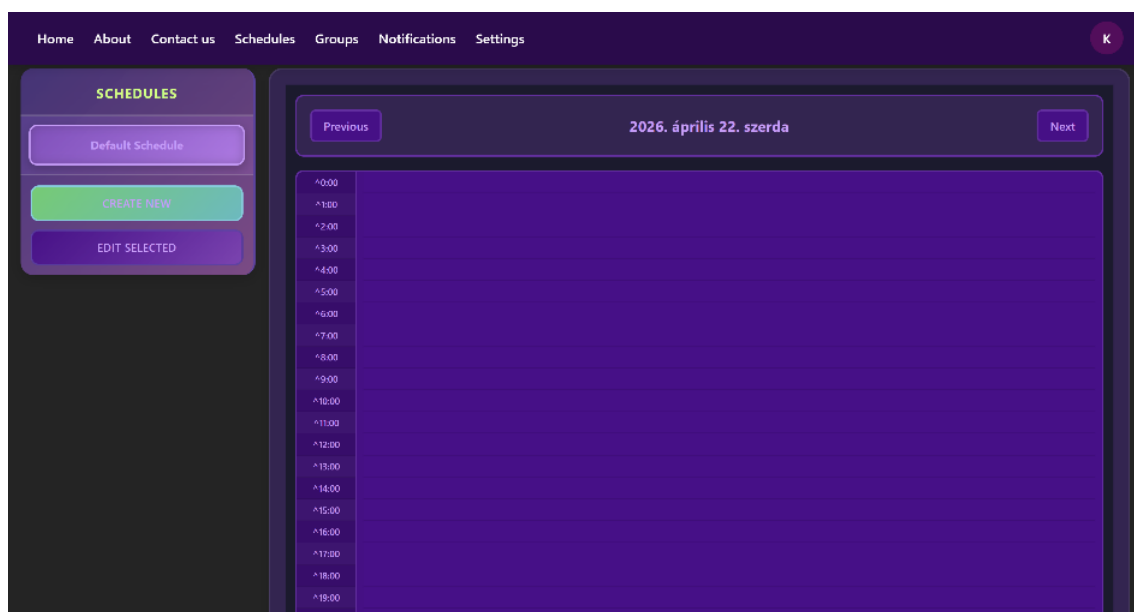


11. képleírás: A heti elrendezésű naptár előnézete

¹⁰ Grid: Táblázatalapú, sorokból és oszlopokból álló elrendezés, a responszivitás megvalósítását segítő elrendezési forma

CalendarDayView.jsx

Ez a komponens felelős a napi felbontású naptár megjelenítéséért és minden vele kapcsolatos számításért. Megjeleníti a kiválasztott napot és az adott naphoz kapcsolódó blokkokat oszlopos elrendezésben. A heti nézethez hasonló stílusban jeleníti meg az adatokat. Az oszlop tetején a dátumot és napot kijelző fejléc található. Arra kattintva vissza tudunk térni a mai napi dátumhoz. A napot kijelző oszlop bal oldalán – a heti nézethez hasonlóan – a vizualizálást elősegítő oszlop található, amely az órákat tartalmazza. Ezek a címkék méretarányosan helyezkednek el, tehát a mellette látható blokkok pontos ideje meghatározható.



12. képleírás: A naptár napi elrendezésének előnézete

CalendarView.jsx

Ez a komponens felelős a naptárnézet minden fő műveletének és az oldal elrendezésének megvalósításáért. A komponens a továbbiakban még a „Fetch API”¹¹ hívásokkal mind a naptárakkal kapcsolatos műveleteket, mind a blokkokkal kapcsolatos műveleteket elmenti. Ezekhez a műveletekhez a „selectedSchedule”¹² „useState”¹³ változó tartja számon azt a kiválasztott naptárat, amelyen jelenleg a változtatásainkat kívánjuk végezni. Az „events”¹⁴ „useState” számon tartja a jelenleg kiválasztott intervallumon belüli blokkokat, amelyeket minden komponens render¹⁵ után újratölt a „FromDate” és „ToDate” változók segítségével. Ezek a változók tartalmazzák a képernyőn jelenleg látható intervallum kezdeti- és végdátumát. Ezeken felül a komponens még felelős a naptár műveletek felugró ablakainak megjelenítéséért, és azok műveleteinek végrehajtásáért is. A komponens egy konténer¹⁶ elem, amely a különböző nézetekhez egységesen tárolja/kezeli az adatokat.

GroupSelector.jsx

Ez a komponens felelős a felhasználók csoportjaiért, és az azokkal kapcsolatos információk megjelenítéséért. Az első rész különböző általános információkat jelenít meg a felhasználóval kapcsolatban, mint például: hány aktív csoporttal rendelkezik vagy az adott csoportokon belül hány különböző naptár található. A második rész egy kártyaszerű elrendezésben jeleníti meg a felhasználó csoportjait. Ezek az adott képernyő méretétől függően töltik ki a rendelkezésükre álló teret, és oszlopos formára váltanak, amennyiben az adott kártyák nem férnének ki. A kártyákon látható a csoport neve, a profilképe, amely a csoport nevének kezdőbetűiből áll össze, és az ezekhez tartozó különböző műveletek elvégzéséhez kapcsolatos gombok. A gombok egy saját felugró ablakot jelenítenek meg. Az „Invite”¹⁷ gomb, egy szimpla párbeszéd ablakot jelenít meg, amely egy e-mail címet kér bemenetként. Ha létezik olyan felhasználó, aki a megadott e-mail címmel rendelkezik, akkor egy meghívót küld az adott személynek a kiválasztott csoporthoz. A második gomb a csoporttal kapcsolatos szer-

¹¹ Fetch API: Egy modern JavaScript felület, amely lehetővé teszi a hálózati kérések egyszerű és ígéreteken (Promise) alapuló kezelését

¹² selectedSchedule: Egy useState változó, amely a jelenleg kiválasztott naptár objektumát tárolja

¹³ useState: A React egy alapvető hook-ja, amely a komponenseken belüli állapot tárolását teszi lehetővé

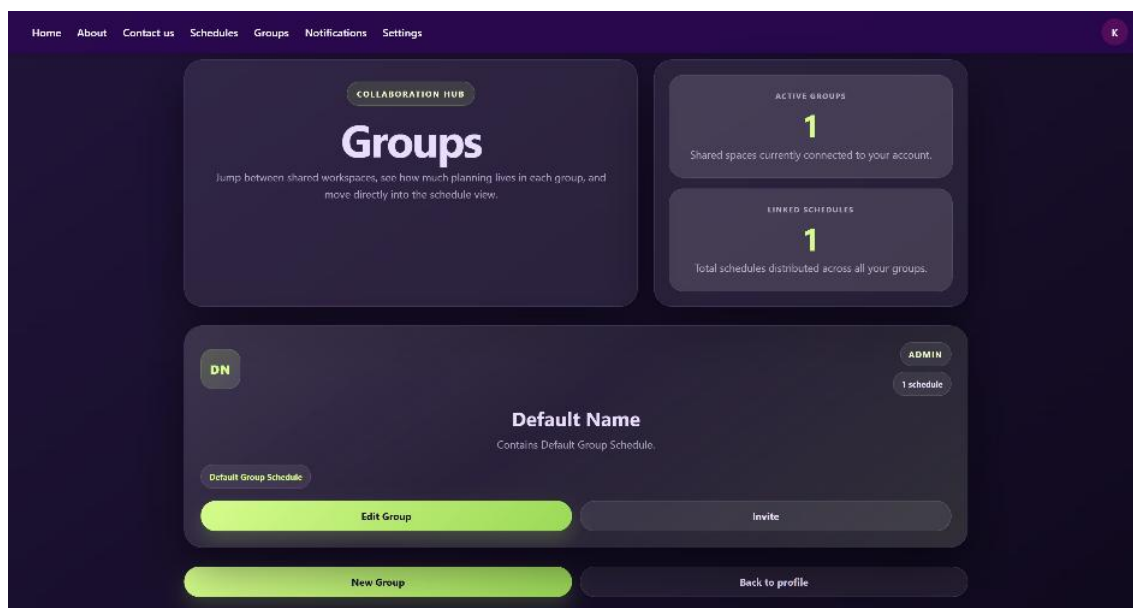
¹⁴ events: Egy listát tartalmazó useState, amely minden olyan blokkot tartalmaz, amelyre jelenleg a Frontend-nek szüksége van

¹⁵ render: Egyes komponensek először betöltése vagy bármely a komponens állapotát tartalmazó mező változásakor lefutó újra töltés, amely már tartalmazza a frissített adatokat

¹⁶ konténer: Egy olyan elem, amely főként adatokat és a magába ágyazott egyéb komponenseket kezel

¹⁷ Invite: A gombra kattintáskor a csoportokhoz meghívó küldéséért felugró ablakot jeleníti meg

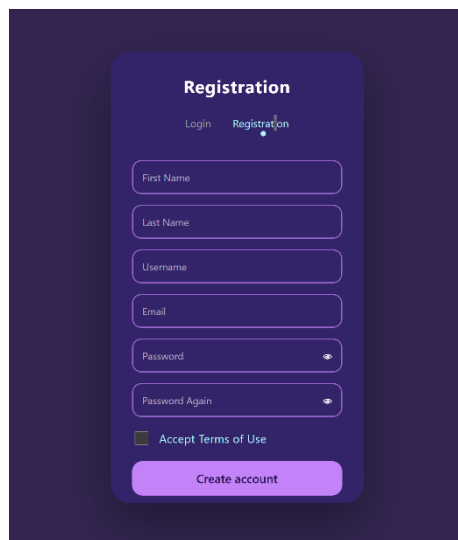
kesztési lehetőségeket tartalmazó felugró ablakot jeleníti meg. Ebben az ablakban elvégezhethetjük a csoport metaadataival kapcsolatos változtatásokat, mint például a leírás vagy név-változtatás. A felugró ablak középső szekciójában a csoportban lévő felhasználók jogosultságait állíthatjuk be. Amennyiben a csoport túl sok felhasználóval rendelkezik a megjelenítéshez, az egyesével történő felsorolás helyett egy legördülő listává vált át. Maga a jogosultságot beállító mező is egy legördülő lista, amely tartalmazza a jogosultságokat, valamint egy extra „Remove” opciót. Az utóbbi az adott felhasználó csoportból való eltávolításáért felelős. Ha magukra a csoportokat tartalmazó kártyákra kattintunk, akkor a „Schedules” komponens újra felhasználhatóságának köszönhetően az adott csoport és naptárainak megjelenítő nézetére válthatunk. Az egyetlen változás a „Schedules” komponenssel kapcsolatban csak a háttérben zajlik. Itt a saját felhasználónkhoz kapcsolódó naptárak helyett a csoport naptárait tölti be és menti el a naptárakat tartalmazó „UseState”-be.



13. képleírás: A "Groups" oldal előnézete

LoginModule.jsx

Ez a modul felelős a felhasználó manuális beléptetéséért vagy regisztrációjáért. A komponensen két fő egységet találunk. Ezeket a fejlécben található „Login”¹⁸ és „Registration”¹⁹ szavakra kattintva váltogathatjuk. A „Login” részén a komponensnek két egyszerű beviteli mező található, amely egy e-mail címet és egy jelszót kér tőlünk. Ezek megadását követően a belépés gombra kattintva egy „Fetch API” kérés segítségével elküldi az adatainkat a „Backend”-nek, ahol a jelszavunk ellenőrzését követően kapunk egy választ. Ez a válasz a token, amellyel a további kéréseket végezhetjük majd ha sikeres; vagy egy egyszerű szöveges üzenet – amely tájékoztat a fellépő hibával kapcsolatban – ha sikertelen. A komponens másik fő részén a „Registration”-on a még felhasználói fiókkal nem rendelkező kliensek tudnak regisztrálni. Az itt lévő mezők kitöltése után a komponens egy validációt²⁰ futtat le, amely ellenőrzi: megfelelő formátumú-e az email-cím; nem tartalmaz-e a név számokat; elfogadta-e a felhasználó a felhasználói feltételeket; elég erős jelszót adtak-e meg. Amennyiben ezek mindegyike megfelelő, létrehoz egy felhasználói fiókot az alapértelmezett adatokkal egy „Fetch API” hívás segítségével, majd azt követően egyszerre be is lépteti a felhasználót a fiókjába. Amennyiben azonban valamely mező értéke érvénytelen, akkor a komponens a bemeneti mezők alján, egy hibaüzenet formájában közli a felhasználóval, hogy a kitöltött értékek mely feltételeknek nem felelnek meg.



14. képleírás: A regisztráció előnézete

¹⁸ Login: Bejelentkezés már meglévő felhasználói fiókba

¹⁹ Registration: Új felhasználói fiók létrehozása

²⁰ validáció: Egy ellenőrző függvény, amely átnézi az összes mezőt, hogy megfelelő értékeket tartalmaznak-e

NavModule.jsx

Ez a komponens felelős a navigációs mező tartalmának és hivatkozásainak megjelenítéséért és kinézetéért. Minden oldalon jelen van, kivéve a bejelentkezett felhasználó főoldalán, ahol a hivatkozások már menüpontok formájában is elérhetőek. Ezen ok miatt a navigációs menü megjelenítése felesleges redundancia²¹ lenne. A navigációs menü egy fontos „property”-vel rendelkezik, amely segít eldönteni, hogy a két kinézeti verzió közül melyik kerüljön megjelenítésre. Ez a mező egy igaz/hamis értéket tárol az alapján, hogy jelenleg egy bejelentkezett felhasználó felületét jeleníti-e meg. Amennyiben a mező értéke igaz, akkor a bővített, már az egyes funkciókat takaró menüpontokat is megjeleníti a hivatkozások listájában. Amennyiben nincs bejelentkezett felhasználó, akkor a rövid, statikus információkat leíró oldalak hivatkozásai jelennek meg a menüpontok közt. A navigációs menü jobb oldalán egy kis profil ikon található. Az ikon alapértelmezetten egy „B” betűt tartalmaz, erre kattintva átlépünk a bejelentkezési oldalra. Ha a felületen egy bejelentkezett felhasználó található, abban az esetben viszont az adott felhasználó kezdőbetűit tartalmazza. Az ikonra kattintva itt is visszatérhetünk a bejelentkezett felület kezdőoldalára.

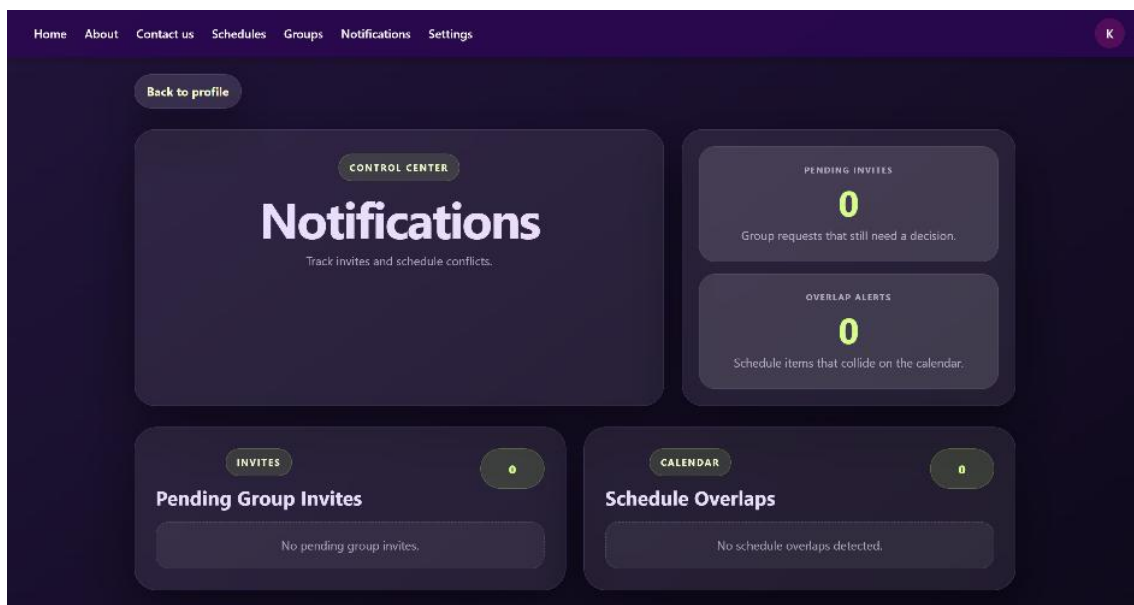
ColorsAndFonts.jsx

Ez a komponens egy rejtett hivatkozáshoz tartozik. Ezzel végeztünk különböző tesztelési és kinézeti beállításokat az oldal fejlesztése során. Az oldalt a „/dev” Url segítségével tudjuk elérni. A végleges leadott projektben a különböző felülíró funkciók nélkül, már csak a sablonokat és színeket tartalmazva van jelen.

²¹ redundancia: felesleges ismétlődés/többlet

NotificationComponent.jsx

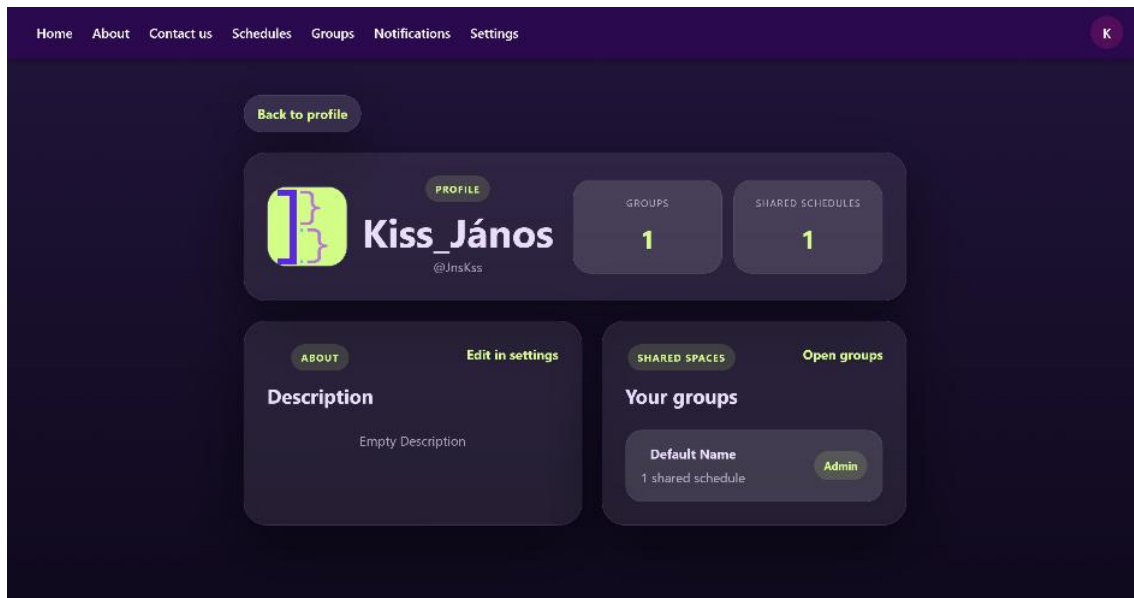
Ez a komponens felelős az értesítéseinkkel kapcsolatos információk megjelenítéséért és kezeléséért. Három fő részből áll. Az első részben egy statisztikai szekció található, amely különböző általános információkat jelenít meg az értesítéseinkről. Ezek közé tartozik például, hogy hány egymást fedő blokkal rendelkezünk vagy, hogy hány még nem elfogadott meghívója van a felhasználónak. A másik két blokk a képernyő méretétől függően két oszlopban, vagy amennyiben nem férne ki, egymás alatt jelenik meg. Az első oszlop a meghívóinkkal kapcsolatos információt tartalmazza. Itt láthatjuk a jelenleg még függőben lévő meghívóinkat, amelyeket elfogadhatunk vagy visszautasíthatunk. Mindkét esetben a választásunk azonnal elmentésre kerül egy „Fetch API” hívás segítségével a „Backend”-en. A másik oszlop a konfliktálódó blokkokat és azok információit jeleníti meg. Itt az adott blokkokat, amelyek fedik egymást dátum alapján csoportosítva látjuk a blokk pontos idejével és nevével együtt. Ezen kívül a blokkokat reprezentáló egységeken található egy gomb, amellyel a felhasználó eldöntheti, hogy ignorálni kívánja-e az adott blokkra vonatkozó értesítéseket vagy sem. Ennek mentése szintén egy „Fetch API” hívás segítségével történik a „Backend”-re. Minden egyes személy saját maga döntheti el, hogy ignorál-e egy blokkot vagy sem, ezzel lehetővé téve, hogy a csoportban mindenki számára külön beállítással rendelkezzen ugyanazon blokk.



15. képleírás: A "Notifications" oldal előnézete

ProfileDescriptionModule.jsx

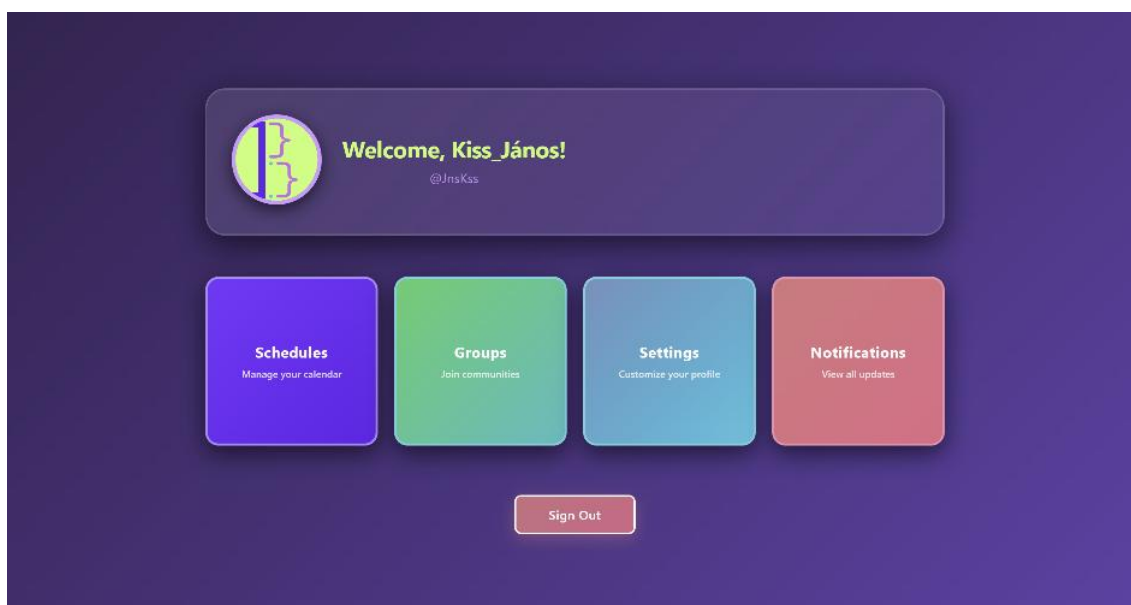
Ez a komponens felelős a felhasználói profilunk részletes megjelenítéséért. Itt a profilunk adatait és egyéb, statisztikai információkat tudunk megtekinteni. A felső sorban a nevünk, a profilképünk, a jelenleg aktív csoportjainknak száma, és a megosztott naptáraink száma látható. A további részek a képernyő méretétől függően több vagy egy oszlopban jelennek meg. Az első oszlop tartalmazza a felhasználói profilunk leírását. A második oszlop, pedig a csoportjainkkal kapcsolatos publikus információkat jeleníti meg. Itt megtekinthetjük az aktív csoportjainkat, azok nevét, és az adott csoportokhoz tartozó jogosultságainkat. Ezeken kívül minden információs blokk jobb felső sarka tartalmaz egy „ugrás a ...-ra” gombot, amely az adott szekcióhoz tartozó aloldalra irányít át minket. Ezt a komponenst a későbbiekben tervezzük tovább fejleszteni és újra felhasználni a felhasználók közti böngészés funkcióra.



16. képleírás: A profilokat részletesen megjelenítő oldal előnézete

ProfileModule.jsx

Ez a fő oldal, amely a bejelentkezést követően várja a felhasználót. Itt található egy rövidített, a profilunkat leíró szekció, amelynél a profilképre kattintva a felhasználót részletező nézetre válthatunk. Ezek mellett, ennek a mezőnek a jobb szélén, ha a felhasználó rendelkezik aktív értesítésekkel, akkor azt egy felkiáltójel jelzi. Erre a felkiáltó jelre kattintva az értesítéseinket megjelenítő nézetre tudunk átvágni. Az alsó részén az oldalnak egy gomb panel található. Ezek a gombok a szoftver különböző funkcióit reprezentáló aloldalakra léptetnek át kattintáskor.

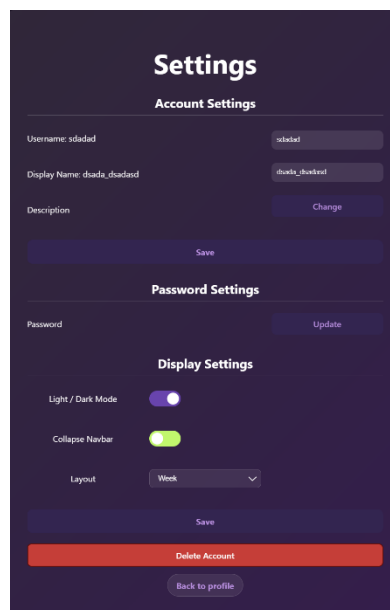


17. képleírás: A bejelentkezett felhasználók főoldalának előnézete

SettingsModule.jsx

Ez a komponens felelős a különböző kinézeti- és felhasználói adatok megjelenítéséért, valamint kezeléséért valamely változtatás esetén. A beállítások külön csoportokra vannak bontva a későbbiekben lehetővé téve az egyszerű kategorizált bővítést. Az első csoport a felhasználó személyes adatainak változtatására nyújt lehetőséget. Itt a felhasználónk meg tudja változtatni a felhasználónevét, a megjelenített nevét és leírását. A leírás a terjedelmét tekintve egy saját felugró ablakkal rendelkezik a kényelem érdekében. A következő kategória a jelszó-változtatási opciót tartalmazza. Itt a jelszóváltoztatás gombra kattintva kapunk egy felugró ablakot, ahová a jelenlegi jelszavunkat, majd az újonnan beállítani kívánt jelszót kell megadni kétszer. Ezt követően a mentés gombra kattintva – amennyiben helyes jelenlegi jelszót adtuk meg –, a „Backend” elmenti az újonnan megadott jelszavunkat. A jelenlegi utolsó beállítási kategória a különféle egyénileg beállítható megjelenítési beállításokat tartalmazza. Itt változtathatjuk az oldal színeinek témáját, hogy a navigációs menü összezsukható legyen,

valamint a naptár nézet intervallumának méretét. Ez a beállítási lehetőség adja meg, hogy a naptárunkat havi/heti/napi felosztásban kívánjuk-e megtekinteni. Minden csoportot külön lehet elmenteni annak érdekében, ha nem a teljes beállításunkat szeretnénk módosítani és menteni. Amennyiben változtatunk, de nem mentjük el, akkor az oldal automatikusan visszaállítja a beállításainkat arra, ami a változtatások előtt volt. Ezt azért is teszi, mert egyes beállításokat a „localStorage”-ba is elment a gyorsabb betöltési idők érdekében. Az utolsó funkció, amelyet ez a komponens nyújt, az a felhasználói profilunk, és az ehhez kapcsolódó minden adat törlése. A gombra kattintva egy felugró ablak jelenik meg, amely egy megerősítő ablakként szolgál annak érdekében, hogy biztos törölni szeretnénk-e a fiókunkat.



18. képleírás: A "Settings" oldal előnézete

App.jsx

Ez a komponens a „React” keretrendszer alapvető komponense, amelyen belül beágyazzuk a többi általunk készített modult. Ez felelős továbbá a „localStorage”-ban lévő információ ellenőrzéséért. Amennyiben a „localStorage” tartalmaz egy API-tokent, akkor ez egy automatikus bejelentkezési kísérletet próbál meg. Ha ez sikeres, akkor a főoldal helyett a felhasználót a saját főoldala fogadja. Amennyiben viszont a felhasználói adatok lekérése sikertelen, akkor a korábbi „localStorage”-ba betöltött adatokat is kitörli. Ezekon felül ebben a komponensben található a „react-router”. Ez a beimportált csomag kezeli a „React” rendszeren belüli URL alapú hivatkozásokat. Itt attól függően, hogy a felhasználó be van-e jelentkezve – amelyet egy „useState” tart számon – meghatározza az elérhető hivatkozásokat, az azokhoz tartozó betöltendő komponenseket, és még a navigációs menü formátumát is. A

formátum meghatározásához egy „property” tulajdonság mezőt tölt ki a komponens. Ez egy igaz/hamis mező, amely eldönti, hogy a hosszabb vagy rövidebb formátumra van szükségünk. Ezeken felül ez tartalmazza az API hívó osztály „Singleton”²² példányát, amelyeket szintén „property”-ken keresztül teszünk elérhetővé minden olyan komponens számára, amelynek szüksége van rá, ezzel lényegesen megkönnyítve a „Fetch API” használatát. De nem csak ezt az adatot kezeli. A másik, amely szintén egy „Singleton” globális szinten elérhető objektum: a „UserData”, amely tartalmazza a felhasználókkal kapcsolatos összes fontos információt, amelyre az oldalnak szüksége van a működéshez. Ezeken felül még különböző dekorációs elemeket is kezel, mint például a klikkeléskor történő effektért felelős komponens.

A komponenseket segítő szkript fájlok:

call-api.js:

Ez a szkriptfájl a „Fetch API” hívások megkönnyítéséért felelős. Az egész kód egy osztály, amelynek egyes mezői a különböző adatok kezelését végzik. Az osztály számon tartja az API alap URL-jét, és a tokenünket, hogy ne kelljen minden hívásnál ezt manuálisan megadnunk. Ezek mellett rendelkezik egy aszinkron hívás mindkét JavaScript-es formájával (Await/Then láncolat). Ezek egy-egy saját függvénnyel rendelkeznek. A függvényeknek 5 fő bemeneti értéke van. Az első érték egy szöveges érték, amely az API végpontjának URL-jét kéri bemenetként. A második az a metódust meghatározó, szintén szöveges beviteli mező. Ez maga a hívás felépítésénél fogja beállítani a „method” mezőt. A harmadik bemeneti érték a hívás során elküldendő üzenet törzse. Ezt egy objektum vagy lista formájában kéri, amelyet a metódus a híváskor átalakít egy általános, API híváshoz szükséges JSON formátumra. A negyedik bemeneti mező egy igaz/hamis értéket vár, amely azt határozza meg, hogy a válaszul kapott „Response” objektumot JSON formátumban (ha igaz) vagy szöveges formátumban kérjük-e vissza (ha hamis). Ezt nem kötelező megadnunk. Alapértelmezetten igaz értékkel rendelkezik, azaz JSON formátumra konvertálja át a válaszul kapott objektumot. Az utolsó, szintén nem kötelezően megadandó mező az extra URL-ben kívánt paramétereket kéri értékül „/” jelekkel elválasztott szöveg formájában. Ezt a mezőt ha elhagyjuk, akkor a metódus semmilyen extra paraméter értéket nem csatol az URL-hez az API hívás elvégzésénél. Amennyiben mégis megadunk egy értéket, ezt automatikusan az URL-ünk után fűzi. A függvény visszatérési értéke a válaszul küldött üzenet/objektum a kiválasztott formában.

²² Singleton: Csak egy példányban létező objektum megnevezése

Ha a függvény a hívás során valamilyen hibát érzékel vagy válaszul egy hibaüzenetet kap a „Backend”-től, akkor a beépített hibakezelő ezt a hibát a konzolon megjeleníti és visszatér a hibaüzenettel.

UserClass.js:

Ez az osztály a UserDataClass-ban történő tárolást segíti elő. Ez az osztály csak az adott felhasználóra vonatkozó fontosabb információkat tárolja. Itt kerül elmentésre a felhasználónév, a megjelenített név, az E-mail cím, és egyéb általános felhasználói adatok. A profilkép URL-je is szerepel a konstruktor bemeneti értékei között, de azt a szoftver jelenleg nem használja sehol, ezért a felhasználók profilképe egy előre meghatározott, alapértelmezett profilkép. A mező viszont lehetőséget ad a későbbiekben történő továbbfejlesztésre.

UserDataClass.js:

Ez az osztály felelős a felhasználónkkal kapcsolatos összes fontos adat tárolásáért és az értesítések megkereséséért. Az utóbbit egy függvény intézi, amely átnézi, hogy a felhasználónk rendelkezik-e függő meghívóval. Amennyiben nem, elvégez egy „Fetch API” hívást is, amely lekérdezi az összes egymást fedő esemény blokkját a „Backend”-ről. Ha bármelyike ezeknek a feltételeknek rendelkezik legalább egy elemmel, akkor ez a függvény igaz értéket ad vissza. Ezt az értéket a bejelentkezett felhasználó főoldalán alkalmazzuk. Amennyiben a visszakapott érték igaz, akkor az oldal egy piros felkiáltójel megjelenítésével jelzi számunkra, hogy aktív értesítéssel rendelkezünk. Az adatok tárolásával kapcsolatban is elvégez néhány, a struktúrát egyszerűbbé tevő szűrést. A kapcsolótáblákat és adataikat kihagyja, és helyette a már összekapcsolt objektumokat tárolja el. Külön mezőkbe válogatja az olyan csoportokat, amelyekhez még nem tartozunk, csak meghívóval rendelkezünk hozzájuk. A felhasználó adatainak tárolásához pedig a már meglévő „UserClass”-t használja.

Backend:

A háttérben futó szolgáltatás, amely felveszi a kapcsolatot az adatbázissal, majd ezt leképezi a Modell mappában lévő sablon osztályokra. Ezek az osztályok „EntityFramework”²³ és a címkék segítségével kiolvassák és eltárolják az adatainkat az egyes osztályokból álló „DbSet”²⁴-ekbe példányok formájában. De nem csak az adatbázissal, hanem a „Frontend”²⁵-el is kapcsolatot létesít. Innen fogadja a „Fetch API” kéréseit és feldolgozza azokat, amennyiben a bemeneti értékek, az üzenet törzse, és az azonosítási mező megfelelően van kitöltve. Ezt követően visszatér egy értékkel, amely a keresett adatokat tartalmazza vagy egy hibaüzenetet. Természetesen nem csak olvasási lehetőséget, hanem az adatbázis mezőinek módosítását és törlését is lehetővé teszi különböző végpontokon keresztül.

Végpontok és rövid leírásaik:

Az egyes példányokat kezelő Controllerek:

Minden kontroller az általa kezelt példányok minden módszerével rendelkezik.

- GET A válasz az összes adatot tartalmazó lista
- GETBYID(„id”) A válasz a keresett azonosítójú példányt tartalmazó objektum
- POST A kérésben kapott objektumot létrehozza
- PUT(„id”) A kérésben kapott objektumot frissíti az azonosítója alapján
- DELETE(„id”) A kérésben kapott azonosítójú elemet törli

Ezekhez a metódusokhoz csak az Admin²⁶ felhasználók férnek hozzá, de jelenleg a „Frontend” része a programunknak nem rendelkezik egy kifejezett Admin felülettel.

AdvancedInfoController:

Ez a kontroller felelős az átlag felhasználók különböző műveleteinek kiszolgálásáért.

- GET Get(„token”)
Ez a végpont visszaadja a felhasználónkkal kapcsolatos összes fontos információt.
- GET Members/”(token”)/(groupId)

²³ EntityFramework: Egy .NET alapú ORM(Object-Relation-Mapping) keretrendszer, amely SQL lekérdezések írása nélkül lehetővé teszi az adatbázis objektumokon keresztül kezelését

²⁴ DbSet: Az EntityFramework-on belül az adatbázis egy adott tábláját leképező objektum

²⁵ Frontend: Webes alkalmazás (Weboldal), amely a felhasználó által közvetlenül látható, és használt része. A megjelenítésért, az interakciókért és a felhasználói élményért felel

²⁶ Admin: Rendszergazda jogosultságú felhasználó

Ez a végpont attól függően, hogy az adott csoporthoz rendelkezünk-e hozzáféréssel, visszatér a tagok listáját.

- GET OverLaps/(„token”)

Ez a végpont átvizsgálja a felhasználónk összes esemény blokkját, és amennyiben talál olyan időpontokat, amelyek fedik egymást, dátum szerint csoportosítva visszatér azokat.

- GET BlocksInRange/(„token”)/(„scheduleId”)/(„from”)/(„to”)

Ez a végpont, amennyiben a felhasználó rendelkezik a megfelelő kapcsolattal/jogosultsággal a megadott naptárhoz, az ahhoz tartozó blokkok közül visszatéríti az intervallumon belül eső események listáját.

- POST groupInvite/(„token”)/(„groupId”)/(„emailToInvite”)

Ez a végpont felelős az új meghívók létrehozásáért, amennyiben a meghívást kezdeményező fél rendelkezik a megfelelő jogosultságokkal.

- POST scheduleCreate/(„token”)/(„groupId”)

Ez a végpont felelős egy új naptár létrehozásáért egy csoport alatt, amennyiben a kérést küldő fél a megfelelő jogosultságokkal rendelkezik.

- POST blockCreate/(„token”)/(„scheduleId”)

Ez a végpont felelős az új blokkok létrehozásáért az adott naptárhoz, amennyiben a kérést küldő felhasználó rendelkezik hozzáféréssel az adott naptár szerkesztéséhez.

- POST userCreate/(„token”)

Ez a végpont felelős az új egyéni naptárak létrehozásáért.

- POST GroupUserConn/(„token”)/(„groupId”)

Ez a végpont felelős a folyamatban lévő meghívók elfogadásának/elutasításának kezeléséért.

- PUT EditGroup/(„token”)/(„groupId”)/(„groupName”)

Ez a végpont felelős a csoportok adatainak szerkesztéséért, ha a kezdeményező fél a megfelelő jogosultságokkal rendelkezik.

- PUT scheduleUpdate/(„token”)/(„groupId”)/(„scheduleId”)

Ez a végpont felelős a naptárak adatainak frissítéséért.

- PUT userUpdate/(„token”)/(„scheduleId”)

Ez a végpont felelős a felhasználói adatok frissítéséért, főként a jelszóváltoztatásért, mely esetén az új jelszót ismét titkosítja és elmenti.

- PUT blockUpdate/(„token”)/(„scheduleId”)/(„blockId”)

Ez a végpont felelős a blokkok adatainak frissítéséért, amennyiben a kezdeményező fél rendelkezik a megfelelő jogosultságokkal.

- DELETE DeleteBlock/(„token”)/(„scheduleId”)/(„blockId”)

Ez a végpont felelős a megadott blokk törlésének elvégzéséért, amennyiben a kezdeményező fél a megfelelő jogosultságokkal rendelkezik.

- DELETE DeleteGroup/(„token”)/(„groupId”)

Ez a végpont felelős a csoportok törlésének elvégzéséért, amennyiben a kezdeményező fél a megfelelő jogosultságokkal rendelkezik.

- DELETE DeleteSchedule/(„token”)/(„scheduleId”)

Ez a végpont felelős a naptárak törlésének elvégzéséért, amennyiben a kezdeményező fél a megfelelő jogosultságokkal rendelkezik.

UserHasAccessToSchedule segédmetódus:

Ez a metódus a különböző módosításokkal kapcsolatos lekérdezéseket segíti. A bemeneti értékeként egy „User” objektumot, és egy naptár azonosítóját várja. Ezt követően egy kételemű tömbbel tér vissza a függvény. A tömb mindkét értéke egy igaz/hamis mező. Az első érték a felhasználó személyes szintű kötődéséről, a második érték a felhasználó csoportokon keresztüli kapcsolatáról nyújt visszajelzést az ezt használó végpontoknak.

LoginController:

Ennek a kontrollernek a „POST” metódusa felelős a felhasználók bejelentkeztetéséért. A végpont a törzsben kapott e-mail címet és jelszót ellenőrzi, majd azt követően, ha ez megfelelő, akkor üzenetben visszaküldi a felhasználónak személyesen előállított token, amelyet a későbbiekben minden „Fetch API” hívás során el kell küldenie. Ez a karakterlánc tartalmazza a felhasználók szerepkörét és a személyazonosságának igazolásához szükséges információkat. Ezek mellett az „AdvancedInfo” végpont tartalmaz olyan metódusokat, amelyek a tokenünket bemeneti paraméterként várják.

UsersController:

Ez a kontroller az átlagos felhasználók számára egyetlen különleges végponttal rendelkezik: a „POST” végpontja, amelynek használatához nincs szükségünk a személyazonosságunkat igazoló tokenre. Ezt a végpontot használja a regisztrációs panel a „Frontend”-en, és lehetővé

teszi, hogy a felhasználó létrejöjjön az alapadatokkal. Ezután a manuális/automatikus belépést követően már a „Login” végpontot használjuk, amely pedig felruház minket a személyazonosságot igazoló tokennel.

Segédosztályok:

TokenManager:

Ez a segédosztály a felhasználók „JWT”²⁷ tokenjeinek generálásáért felelős, amelyeket eltárol az adatbázisban is. A token tartalmazza a felhasználók e-mail címét, és szerepkörét, amelyhez a kapcsolódó jogosultságokat az API konfigurációs fájljából olvassa ki. Ezek a tokenek egy meghatározott időn belül lejárnak, ezáltal inaktívvá téve főként a régóta nem használt tokeneket. Ezt a segédosztályt a 13. évfolyam óráin tanult alapot felhasználva készítettük el. Ezek mellett kibővítettük az osztályt egy metódussal a tokenből való információ könnyebb és gyorsabb kezelésének érdekében. Ez a metódus statikus, egy token típusú objektumra történő meghíváskor az adott tokenbe ágyazott e-mail címet adja vissza. Ezeket főként az „AdvancedInfo” végpont metódusainál alkalmazzuk.

PasswordHandler:

Ez a segédosztály a jelszavak ellenőrzését és titkosítását végzi. Az osztályt szintén a 13. évfolyam óráin tanultakat alapul véve készítettük el a felhasználók jelszavainak kezeléséhez. Az osztály a „Konscius”²⁸ NuGet csomagot használva végzi el a jelszavak titkosítását.

²⁷ JWT: (JSON Web Token) szabványos, tömör, azonosításra és adatátvitelre alkalmazzák

²⁸ Konscius: a jelszavak titkosított formájának biztonságos tárolását, generálását és kezelését végzi

A szoftver használatának rövid bemutatása

Oldalak:

Főoldal:

Ez az első oldal, amelyet egy új felhasználó lát. Itt a szoftver szlogenje és logója látható, valamint a karusszel, mely a szoftver egyes funkcióiról tartalmaz képeket. Ezeken felül a további részekben olvashatunk többet magáról a szoftver által nyújtott funkciókról és rólunk, a készítőkről. Az oldal legalján a lábléceken olvashatóak a GYIK és a felhasználói feltételek.

- About Us: Egy hosszabb, részletesebb leírás, mint ami a főoldalon található
- Contact Us: Fontosabb információk az elérhetőségeinkkel kapcsolatban

Bejelentkezés/Regisztráció:

A „Get Started” vagy a navigációs menün belül az alap felhasználó nélküli ikonra kattintva átléptet minket a weboldal a bejelentkezési föltre. Ha már rendelkezünk felhasználói fiókkal, akkor a „Login” alatt regisztráció helyett beléphetünk abba. Amennyiben nincs fiókunk a regisztráció alatti űrlap kitöltésével létrehozhatunk egy alap felhasználói fiókot, mellyel hozzáférünk a szoftver által nyújtott funkciókhoz. A későbbiekben az alapbeállítások részén a Settings-en belül tudunk változtatni.

Group:

Ez a modul felelős a felhasználó aktív csoportjaival kapcsolatos dolgok megjelenítéséért. Ezen menüpontra lépve a felső sorban általános adatokat tekinthet meg, mint például azt, hogy hány aktív csoportja van. Ez alatt egy kártyaszerű elrendezésben tekintheti meg csoportjait. A kártyákon van lehetőség az adott csoport adatain és a felhasználók jogosultságain módosításokat végezni. Ha megfelelő jogosultsággal rendelkezünk, még meghívót is küldhetünk más felhasználóknak. A kártyára kattintva megjelenik a „Schedules” komponens, csak a saját naptáraink helyett a csoporthoz tartozókat látjuk. Az itt lévő funkciók működése azonos a saját naptárainkat tartalmazó nézet funkcióival, de módosításokat csak akkor tudunk végezni, ha a csoportban megfelelő jogosultsággal rendelkezünk.

Profile Module:

A belépést követően ez az oldal vár minket. Ez a bejelentkezett felhasználók főoldala. Itt tud a felhasználó átlépni a különböző funkciókat biztosító aloldalakra. Az elérhető menüpontok közé tartoznak:

- **Schedule:** A személyes naptárakat megjelenítő oldal
- **Groups:** A csoportjainkkal kapcsolatos információkat megjelenítő oldal
- **Settings:** A beállításokat megjelenítő oldal
- **Notifications:** Az értesítéseinket megjelenítő oldal
- **Logout:** A felhasználó kijelentkezteséért felelős menüpont

Ezek mellett a profilképre kattintva megnyitható egy, a profilunkat részletező oldal.

Schedule:

Miután a felhasználó a Settings-en belül eldöntötte, hogy melyik nézetet szeretné látni, akkor ezen menüpont alatt meg tudja nézni a naptárait, amelyekben az általa kiválasztott naphoz egy új eseményt fel tud venni. Itt láthatja a már meglévő eseményeit is, amelyeket szabadon szerkeszthet és törölhet. Ezek mellett magát a naptárait is tudja módosítani/törölni vagy akár újakat létrehozni. Nézettől függően percenkénti vagy napi lebontásban jeleníti meg a blokkokat. A heti/napi nézetben a blokkok pontos kezdési és befejezési idejét látjuk oszloposan, míg a havi nézetben az adott napokat, és az azokon a napokon történő eseményeket. A havi nézetben – amennyiben egy napon több mint 3 esemény van –, az első hármat jeleníti meg, és jelzi azt is, hogy vannak olyan események, amelyeket csak az adott napra kattintva tud előhozni.

Settings:

A Settings menüpont alatt a felhasználó a különböző felhasználói és kinézeti beállításaihoz tud hozzáférni, amelyeket beállíthat. A beállítást követően, ha elmenti változtatásait, akkor ezt hozzáköti a felhasználóhoz lehetővé téve ezzel, hogy ha új eszközön jelentkezik be, a beállításai ott is megmaradjanak. A felhasználóval kapcsolatban történő beállítások közé tartozik a felhasználónév, a megjelenített név és a jelszóváltoztatás. Ezek mellett különböző stílusbeállítások is módosíthatóak. Beállítható az oldal témája (dark/light), a navigációs menü megjelenítésének módja és a legfontosabb, a naptárak elrendezése is. Az utóbbi lehetőséget ad napi/heti/havi kinézet beállítására. Az utolsó funkció, amelyet tartalmaz az oldal, az a felhasználói fiók és a hozzá tartozó minden adat törlése.

Notifications:

A Notification menüpont alatt a felhasználó hozzáfér a különböző értesítéseihez és azok beállításaihoz. Az értesítéseknek két fő fajtája van. Az első: a meghívásokkal kapcsolatos értesítések. Ha a felhasználók számára érkezett egy meghívó egy csoporthoz, akkor azt itt tudja megtekinteni és eldöntheti, hogy visszautasítja vagy elfogadja. A másik fő fajtája az értesítéseknek: az ütköző események. Itt listázva láthatjuk dátum alapján csoportokban, ha az adott napon valamely eseményünk ütközik. Ez lehetővé teszi ezek korai észrevételét az egymást fedő időpontokkal kapcsolatban. Azonban, ha nem kívánjuk ezt folyamatos értesítés formájában látni, bekapcsolhatjuk egy-egy adott blokk ignorálását az ütközéseket illetően.

Profile Description Module:

Ez a modul felelős a profil részleteinek és adatainak megjelenítéséért, amelyet csak az adott felhasználó lát.

Teszteléssel kapcsolatos információk:

Tesztelési körülmények:

- Operációs Rendszer: Windows 10/11
- RAM: 8-16 GB
- CPU: Intel(R) i7-1260P/ I5-9400F
- GPU: Integrált/NVIDIA GeForce GTX 1660 Ti

Automatizált tesztelés:

Unit tesztek:

Az unit tesztjeinket a saját API „solution”²⁹-unk részeként egy másik projekt alatt valósítottuk meg. Ez a projekt az összes controllerünk minden metódusáról tartalmaz egy pozitív és egy negatív eredményű tesztet. Ezeken felül azokhoz a kontrollerekhez, amelyek az összes

²⁹ solution: A “Microsoft Visual Studio” által használt központi konfigurációs fájl, amely több projektet fog össze egy egységbe

„CRUD”³⁰ műveletet megvalósítják tartozik egy láncolási tesztet is. Itt a műveleteket sorrendben teszteljük és minden lépés után vizsgáljuk az állapotát. Először létrehozunk egy új elemet, azt lekérdezzük, utána módosítjuk a tartalmát, majd végül töröljük.

Manuális tesztelés:

Müller Bálint:

A „Schedules” komponens és különböző nézeteinek tesztelését, valamint az egységes stílus, színek és elrendezés ellenőrzését végeztem. Az utóbbi esetében nagy hasznomra volt a „React App” által nyújtott „Live preview”. Ezzel a CSS fájlokban végzett módosításaimat azonnal láttam a megjelenített felületen. Több személynek is megmutattam az oldal kinézetét, amelyet követően rengeteg pozitív visszajelzést kaptam. Természetesen érkezett a design esetleges fejlesztéséhez több javaslat is, amelyeket a projekt fejlesztésének folyamata alatt megvalósítottunk. Ezek mellett a munkám során a különböző a „Schedule”-okat megjelenítő komponensek logikai működését is főként én teszteltem.

Fehér Ágoston:

A komponensek által végzett „Fetch API”-val kapcsolatos műveleteket és egyéb, a háttérben zajló logikai részek működését ellenőriztem. Az utóbbiak során számomra a leghasznosabb funkciónak a töréspontok használata bizonyosult. Ez által a kód különböző részein meg tudtam állítani a futtatást, és ellenőrizhettem a pillanatnyi háttérbeli változók értékét. A „Fetch API” hívásokkal kapcsolatos vizsgálódást a böngészőbe épített hálózati kéréseket figyelő funkció segítette, amely nem csak a böngésző által elküldött üzenet adatait részletezte, hanem a válaszul érkezettét is. Ez természetesen a legtöbb esetben még a hiba okát is tartalmazta, amely ez által jelentősen csökkentette a hiba megtalálásának idejét.

Fülöp László András:

A komponensek reszponzív viselkedését és a bejelentkezésért felelős komponens funkcióit ellenőriztem. Főként a reszponzív felület fejlesztése során volt hasznos a manuálisan végzett ellenőrzés számomra, amely sok apróbb hibát fedett fel a megjelenéssel kapcsolatban. Ezeknek az általam felfedezett problémáknak mindegyikét javítani is tudtam szinte azonnal.

³⁰ CRUD: A négy alapvető adatkezelési művelet rövidítése: Create (létrehozás), Read (lekérdezés), Update (módosítás), Delete (törlés)

Tesztelési eredmények:

Az általunk végzett manuális és a szkriptfájlok segítségével elvégzett automatizált tesztek eredménye rendkívüli segítségnek bizonyult a már meglévő funkciók továbbfejlesztése és véglegesítése során. A teszteknek köszönhetően könnyedén megtaláltuk az újonnan felmerülő hibákat, és ezeket javítottuk is. A projekt jelen állapota szerint megfelelően működik, és minden általunk meghatározott automatikus tesztet sikeres eredménnyel tér vissza.

A szoftver letöltésének és elindításának lépései:

Működésének műszaki feltételei:

A program futtatásához az alábbi alkalmazások szükségesek:

- XAMPP Control Panel
- Visual Studio 2022
- NodeJs v2.0.27

Műszaki és rendszerkövetelmények (minimum):

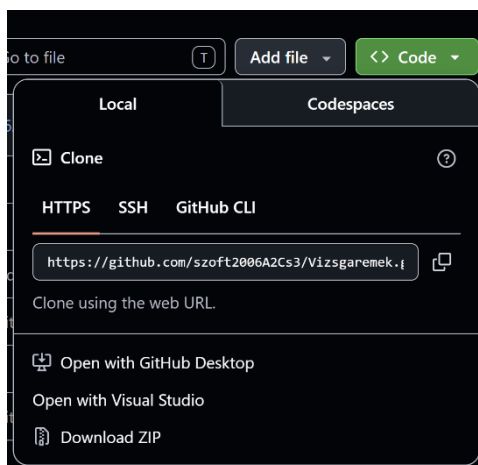
- Processzor: Intel Core i5
- Videókártya: integrált
- Memória: 4GB

A program elindításának részletes leírása lépésről lépésre

1. A „repository”³¹ megnyitása a linken keresztül

³¹ repository: A GitHub-on elérhető konkrétan magát a projektet tartalmazó mappa

2. A zöld „Code >” gomb legördülő menüjéből a letöltés „ZIP”³²-ként gomb választása



19. képleírás: Letöltés a GitHub-ról

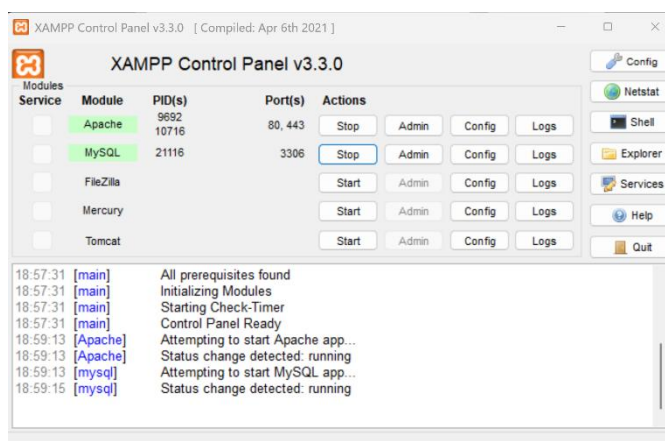
3. A „ZIP” fájl kicsomagolása

Név	Állapot	Módosítás dátuma	Típus	Méret
ColorsAndStyles		2026. 04. 23. 18:56	Fájlmappa	
DataBase		2026. 04. 23. 18:56	Fájlmappa	
Documentation		2026. 04. 23. 18:56	Fájlmappa	
FINAL REACT		2026. 04. 23. 18:56	Fájlmappa	
Plans		2026. 04. 23. 18:56	Fájlmappa	
V2 BackendProjekt		2026. 04. 23. 18:56	Fájlmappa	
README.md		2026. 04. 23. 18:56	Markdown forrásfájl	1 KB

20. képleírás: A kicsomagolt mappa

4. Az adatbázis elindítása:

- a. XAMPP Control Panel „Apache” és „MySQL” szolgáltatásainak elindítása



21. képleírás: XAMPP Control Panel

- b. A „MySQL” admin felület megnyitása

³² ZIP: Egy tömörített fájlformátum

- c. A „DataBase” mappában található „brckettDefaultState” SQL fájl importálása

5. A „Backend” elindítása:

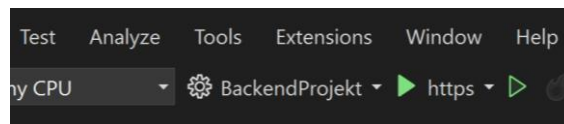
- a. A V2 „BackendProjekt” mappában található „BrckETTAPI.sln” fájl futtatása



Név	Állapot	Módosítás dátuma	Típus	Méret
.vs		2026. 04. 23. 18:56	Fájlmappa	
BrckETTAPI		2026. 04. 23. 18:56	Fájlmappa	
BrckETTAPI.test		2026. 04. 23. 18:56	Fájlmappa	
BrckETTAPI.sln		2026. 04. 23. 18:56	Visual Studio Solu...	2 KB

22. képleírás: A Backend projekt mappája

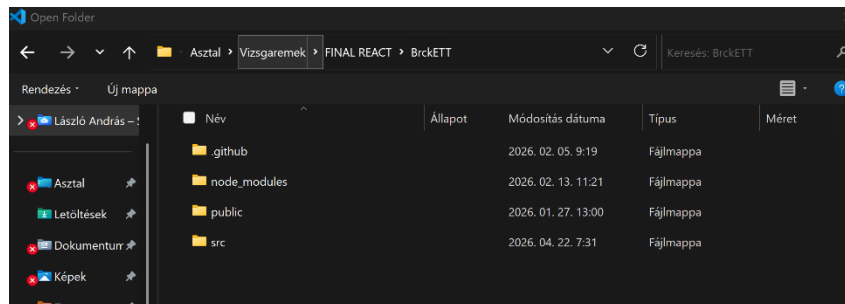
- b. Jobb kattintás a .sln fájlra, majd a NuGet csomagok helyreállítása gombra
- c. A kódszerkesztő megnyílását követően a projekt elindítása Debugger/Release módban



23. képleírás: Debugger/Release mód indítása gomb

A „Frontend” elindítása:

- d. „Visual Studio Code” segítségével megnyitjuk a „FINAL REACT\BrckETT” mappát



24. képleírás: A Frontend projekt mappája

- e. A következő lépésben terminál használatára lesz szükségünk. Ezeket a parancsokat adjuk ki sorban:

- `npm i`

```
idont@DESKTOP-2R5SGBB MINGW64 ~/OneDrive/Desktop/Vizsgaremek/FINAL REACT/BrckETT (main)
○ $ npm i
```

25. képleírás: A csomagok letöltése

- `npm run dev`

```
idont@DESKTOP-2R5SGBB MINGW64 ~/OneDrive/Desktop/Vizsgaremek/FINAL REACT/BrckETT (main)
○ $ npm run dev
```

26. képleírás: A felhasználói felület elindítása

- egy „o” betű beírásával vagy az elérhetőségi hivatkozásra „Ctrl+Click” segítségével tudjuk a programot megnyitni

```
→ Local: http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
o
```

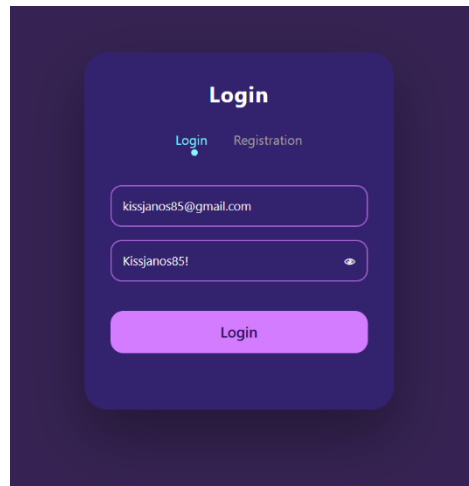
27. képleírás: A felület megnyitása

Bejelentkezés:

Az adatbázisunkat létrehozó importálandó SQL szkriptfájl már tartalmaz egy előre létrehozott felhasználót, amely a program különböző funkcióit demonstráló adatokkal van feltöltve.

E-mail: kissjanos85@gmail.com

Jelszó: Kissjanos85!



28. képleírás: Előre elkészített felhasználó

Továbbfejlesztési lehetőségek

Year View:

A jelenlegi nézet mellé egy éves nézetet is létrehozását tervezzük a jövőben. Ebben a nézetben hónapos lebontásban látni lehet majd, hogy melyik hónapokban van esemény. A hónapokra kattintva megnyitható azok részletező nézete, amelyeken a szokásos, már elérhető funkciókkal tudjuk majd a beosztásokat szerkeszteni.

User Browsing:

A csoportoknál történő meghívásokhoz a későbbiekben hozzá lehet adni egy adott felhasználó profiljának megnyitását, amely a részletező nézetet jelenítené meg, amely az adott felhasználóhoz tartozna.

Admin felület:

Egy külön aloldal, amelyen a BrckETT Admin felhasználói hozzáférnek és kezelni tudják a szoftver adatait. A „Backend” már rendelkezik a megfelelő jogosultságkezeléssel és a megfelelő végpontokkal, csak egy vizuális felület hiányzik a megvalósításához. A tokenek jogosultságai is tartalmazzák az Admin szerepkört. Ezért jelenleg a karbantartási műveleteket akár a XAMPP Control Panelen, akár Swagger³³-en keresztül el tudjuk végezni.

Profilképek feltöltése:

A későbbiekben a jelenlegi struktúrának köszönhetően egy profilkép feltöltési lehetőség könnyen hozzáadható a programhoz. A betöltését illetően a „User” osztály konstruktora már rendelkezik egy jelenleg használaton kívüli „img_Url” bemeneti értékkel.

Csoport ikon feltöltése:

A későbbiekben a profilképekhez hasonlóan könnyen hozzáadható a csoportokhoz egy kép feltöltésének opciója. Az ikon hely már megvan, és kép hiányában már alapértelmezett értékkel is rendelkezik. Amennyiben nincs megadott kép, az ikon a csoport kezdőbetűit tartalmazza. Ezt egyszerűen le lehet a jövőben cserélni egy általunk feltöltött képre, amelyet a csoport szerkesztésekor tudnánk feltölteni.

Backend végpontok paraméterei közti redundancia:

A backend végpontok írásának idején még nem teljesen sajátítottuk el a JWT és API hívások sajátosságait, ezért a fejlécből való kibontás helyett a tokent külön, még egyszer egy paraméterként kérjük be. Ez a hiba könnyen javítható a paraméterváltozók lokálisra történő cseréjével, amelynek tartalma a fejlécből vett token. A kontroller metódusok egyéb működésein nincs szükség változtatásra, csak a „Frontend” „Fetch API” hívások paraméterei közül kell kitörölnünk a tokent tartalmazó szekciókat.

³³ Swagger: REST API-k tervezésére, tesztelésére és dokumentálására szolgáló eszközkészlet

Csoportokból való gyors kilépés:

Jelenleg a csoportokból történő gyors kilépésre nincs lehetőség a programon belül, csak az adott csoport szerkesztési opcióin belül a felhasználói jogosultságok között tudunk kiléptetni felhasználót a „Remove” opcióval. A csoportokhoz tartozó kártya gombjai közé könnyen felvehető egy új gomb, amely a csoportból történő kilépéseket kezelné.

Tartalomjegyzék:

Bevezetés	2
Szoftver célja	2
Felhasználói felületek	2
Készítési folyamat	3
Feladatok felosztása:.....	3
Verziókövetés:	5
Kommunikáció/Csapatmunkához használt egyéb eszközök:	6
Discord:	6
Kijelölések, kommentek:	6
Adatbázis táblái és rövid összefoglalásuk:	7
„blocks” tábla:	7
„groups” tábla:	8
„groupscheduleconn” tábla:.....	8
„groupuserconn” tábla:	8
„isignored” tábla:	8
„schedules” tábla:	8
„scheduleusersconn” tábla:.....	9
„templates” tábla:	9
„templatesblocksconn” tábla:	9
„users” tábla:.....	9
„usersettings” tábla:	9
Szoftveregységek és -komponensek leírásai	11
Frontend:.....	11
CSS:.....	11
Frontend komponensek és feladataik:	12
A komponenseket segítő szkript fájlok:	27

Backend:	29
Végpontok és rövid leírásaik:	29
Az egyes példányokat kezelő Controllerek:	29
Segédosztályok:	32
A szoftver használatának rövid bemutatása.....	33
Teszteléssel kapcsolatos információk:	35
Tesztelési körülmények:	35
Automatizált tesztelés:.....	35
Unit tesztek:.....	35
Manuális tesztelés:.....	36
Müller Bálint:	36
Fehér Ágoston:	36
Fülöp László András:	36
Tesztelési eredmények:.....	37
A szoftver letöltésének és elindításának lépései:.....	37
Működésének műszaki feltételei:	37
A program elindításának részletes leírása lépésről lépésre	37
Bejelentkezés:	41
Továbbfejlesztési lehetőségek	41
Year View:.....	41
User Browsing:	41
Admin felület:.....	42
Profilképek feltöltése:.....	42
Csoport ikon feltöltése:.....	42
Backend végpontok paraméterei közti redundancia:.....	42
Csoportokból való gyors kilépés:	43

Ábrák, Táblázatok és Képleírások:

1. ábra: Az adatbázis ER modellje.....	7
1. táblázat: A feladatelosztások	4
1. képleírás: A GitHub-on végzett munka gráfja.....	5
2. képleírás: A GitHub-on végzett commit-ok egy része	5
3. képleírás: Egy részlet a dokumentumról a készítés folyamán.....	6
4. képleírás: Az oldal alsó része	12
5. képleírás: Az oldal felső része	12
6. képleírás: A "Contact Us" oldal előnézete	13
7. képleírás: Az "About Us" oldal előnézete	13
8. képleírás: Az adott nap eseményeit részletező oldal előnézete	14
9. képleírás: Az új események létrehozását kezelő párbeszédablak.....	15
10. képleírás: A naptár havi elrendezésű előnézete	16
11. képleírás: A heti elrendezésű naptár előnézete.....	17
12. képleírás: A naptár napi elrendezésének előnézete	18
13. képleírás: A "Groups" oldal előnézete.....	20
14. képleírás: A regisztráció előnézete	21
15. képleírás: A "Notifications" oldal előnézete	23
16. képleírás: A profilokat részletesen megjelenítő oldal előnézete	24
17. képleírás: A bejelentkezett felhasználók főoldalának előnézete	25
18. képleírás: A "Settings" oldal előnézete	26
19. képleírás: Letöltés a GitHub-ról	38
20. képleírás: A kicsomagolt mappa.....	38
21. képleírás: XAMPP Control Panel.....	38
22. képleírás: A Backend projekt mappája.....	39
23. képleírás: Debugger/Release mód indítása gomb.....	39
24. képleírás: A Frontend projekt mappája	39
25. képleírás: A csomagok letöltése	40
26. képleírás: A felhasználói felület elindítása.....	40
27. képleírás: A felület megnyitása	40
28. képleírás: Előre elkészített felhasználó.....	41